

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Домашнее задание №3
«Тестирование API средствами Postman»

Выполнил:

Якшин Артемий

Группа БР1.1

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Реализовать тестирование REST API (разработанного в ЛР1 — приложение для бронирования столиков в ресторанах) средствами Postman, написав автоматические тесты внутри Postman. Результат — один комплексный сценарий по основному процессу приложения. По аналогии с примером для сервиса аренды недвижимости (регистрация → вход → список объявлений → фильтрация → просмотр объявления → старт переписки), для данного приложения основной процесс:

**регистрация → вход → профиль → список кухонь → список ресторанов
→ фильтрация по городу → просмотр случайного ресторана →
доступные столики → создание бронирования → просмотр бронирования
→ история бронирований → создание отзыва → отзывы ресторана.**

Ход работы

1. Объект тестирования

Тестируется REST API из ЛР1 (*Node.js + Express + TypeORM + SQLite*), запускаемый локально: `npm run seed` (наполнение тестовыми данными) и `npm run dev`. Базовый адрес — `http://localhost:3000/api/v1`, вынесен в переменную коллекции `base_url` (можно переопределить через окружение Postman или флаг `newman`). В комплект ДЗ входят:

- `Restaurant Booking API.postman_collection.json` — коллекция с 13 запросами и тестами;
- `Restaurant Booking API.postman_environment.json` — окружение с переменной `base_url`.

2. Организация коллекции

На уровне коллекции настроены:

- **Авторизация** — тип `Bearer Token` со значением `{{token}}`; все запросы наследуют её, токен подставляется автоматически после регистрации/входа;
- **Pre-request script** — однократно генерирует данные прогона: уникальный e-mail (`qa_<timestamp>@example.com`), пароль, дату бронирования (на 30–90

дней вперёд) и случайное время из набора слотов — это делает сценарий идемпотентным при повторных запусках;

- **Переменные коллекции** — `token`, `user_id`, `cuisine_id`, `restaurant_id`, `table_id`, `reservation_id`, `review_id` и др., через которые данные передаются от шага к шагу.

3. Комплексный сценарий (13 запросов)

#	Запрос	Проверки (pm.test)
1	POST /auth/register	201; есть <code>token</code> ; есть <code>user</code> с <code>user_id</code> ; e-mail совпадает; нет <code>password_hash</code> . Сохраняет <code>token</code> , <code>user_id</code> .
2	POST /auth/login	200; получен JWT; <code>user_id</code> совпадает. Обновляет <code>token</code> .
3	GET /users/me	200; <code>user_id</code> и <code>email</code> совпадают с зарегистрированным; нет <code>password_hash</code> .
4	GET /cuisines	200; непустой массив; у элемента есть <code>cuisine_id</code> , <code>name</code> . Сохраняет <code>cuisine_id</code> .
5	GET /restaurants	200; постраничный ответ (<code>data</code> , <code>total</code> , <code>page</code> , <code>limit</code>); <code>total</code> > 0; у каждого ресторана есть обязательные поля.
6	GET /restaurants?city=Москва	200; выборка непустая; у всех элементов <code>city</code> == "Москва". Выбирает случайный ресторан из выборки → <code>restaurant_id</code> , <code>restaurant_name</code> .
7	GET /restaurants/{restaurant_id}	200; <code>restaurant_id</code> /name совпадают; есть <code>menu_items</code> , <code>photos</code> , <code>cuisines</code> , <code>reviews_count</code> .
8	GET /restaurants/{restaurant_id}/tables?date&time&guests=2	200; непустой массив; у всех столиков <code>capacity</code> ≥ 2; есть свободный (<code>status</code> == "available"). Сохраняет <code>table_id</code> .
9	POST /reservations	201; есть <code>reservation_id</code> ; <code>status</code> == "confirmed"; <code>guests_count</code> , <code>reservation_date/time</code> , <code>table_id</code> , <code>user_id</code> совпадают. Сохраняет <code>reservation_id</code> .

10	GET /reservations/{{reservation_id}}	200; возвращено именно созданное бронирование; связано с выбранным рестораном.
11	GET /users/me/reservations	200; постраничный ответ; список содержит reservation_id из шага 9.
12	POST /reviews	201; есть review_id; rating == 5; restaurant_id, user.user_id совпадают; непустой comment. Сохраняет review_id.
13	GET /restaurants/{{restaurant_id}}/reviews	200; есть data, average_rating (число), total; список содержит review_id из шага 12.

4. Тесты внутри Postman

Тесты написаны на вкладке *Tests* каждого запроса с использованием API `pm.test()` / `pm.expect()` (Chai). Проверяются: HTTP-статус, структура и типы полей JSON-ответа, бизнес-инварианты (фильтр применён ко всем элементам, бронь подтверждена, отзыв виден в списке и т. п.), а также передача данных между шагами через переменные коллекции. Пример (фрагмент тестов запроса «Создание бронирования»):

```
pm.test('Статус 201 Created', () => pm.response.to.have.status(201));
const r = pm.response.json();
pm.test('Бронирование создано корректно', () => {
  pm.expect(r).to.have.property('reservation_id').that.is.a('number');
  pm.expect(r.status).to.eql('confirmed');
  pm.expect(r.guests_count).to.eql(2);

  pm.expect(r.reservation_date).to.eql(pm.collectionVariables.get('reservation_date'));

  pm.expect(r.table.table_id).to.eql(Number(pm.collectionVariables.get('table_id')));
  pm.expect(r.user_id).to.eql(Number(pm.collectionVariables.get('user_id')));
});
pm.collectionVariables.set('reservation_id', r.reservation_id);
```

Пример выбора случайного ресторана из отфильтрованной выборки (запрос «Фильтрация по городу»):

```
pm.test('Фильтр по городу применён ко всем элементам', () => {
  json.data.forEach(r => pm.expect(r.city).to.eql('Москва'));
});
const idx = Math.floor(Math.random() * json.data.length);
const chosen = json.data[idx];
pm.collectionVariables.set('restaurant_id', chosen.restaurant_id);
pm.collectionVariables.set('restaurant_name', chosen.name);
```

5. Запуск

Сначала поднять API:

```
cd "БР1.1/Якшин Артемий/labs/lab1"
npm install
npm run seed
npm run dev          # http://localhost:3000/api/v1
```

Затем — один из вариантов:

- **Postman GUI:** импортировать коллекцию и окружение → *Collection Runner* → запустить коллекцию целиком (запросы выполняются последовательно сверху вниз);
- **CLI (newman):**

```
cd "БР1.1/Якшин Артемий/homeworks/hw3"
newman run "Restaurant Booking API.postman_collection.json" \
  -e "Restaurant Booking API.postman_environment.json"
```

6. Результаты прогона

Прогон коллекции через newman — все 13 запросов и 37 проверок выполнены успешно:

newman

Restaurant Booking API — ДЗ3 (Якшин Артемий, БР1.1)

- 01. Регистрация пользователя
POST .../api/v1/auth/register [201 Created, 659B, 64ms]
 - ✓ Статус 201 Created
 - ✓ Ответ содержит JWT-токен
 - ✓ Ответ содержит данные пользователя без хеша пароля
- 02. Вход в систему
POST .../api/v1/auth/login [200 OK, 654B, 52ms]
 - ✓ Статус 200 OK
 - ✓ Получен JWT-токен
 - ✓ Авторизован тот же пользователь
- 03. Профиль текущего пользователя
GET .../api/v1/users/me [200 OK, 437B, 1ms]
 - ✓ Статус 200 OK
 - ✓ Профиль принадлежит зарегистрированному пользователю
 - ✓ В профиле нет хеша пароля
- 04. Список типов кухонь
GET .../api/v1/cuisines [200 OK, 498B, 1ms]
 - ✓ Статус 200 OK
 - ✓ Получен непустой список кухонь
- 05. Список всех ресторанов
GET .../api/v1/restaurants [200 OK, 2.09kB, 2ms]
 - ✓ Статус 200 OK
 - ✓ Ответ постраничный и содержит рестораны
 - ✓ Каждый ресторан содержит обязательные поля
- 06. Фильтрация ресторанов по городу
GET .../api/v1/restaurants?city=Москва&limit=50 [200 OK, 1.64kB, 2ms]
 - ✓ Статус 200 OK
 - ✓ Получены рестораны Москвы
 - ✓ Фильтр по городу применён ко всем элементам
 - | 'Выбран ресторан #4 — Самовар'
- 07. Просмотр случайного ресторана
GET .../api/v1/restaurants/4 [200 OK, 1.12kB, 1ms]
 - ✓ Статус 200 OK
 - ✓ Возвращён выбранный ранее ресторан
 - ✓ Детальная карточка содержит меню, фото и кухни
- 08. Доступные столики ресторана
GET .../api/v1/restaurants/4/tables?date=2026-06-21&time=20:30&guests=2 [200 OK, 605B, 2ms]
 - ✓ Статус 200 OK
 - ✓ Получен список столиков
 - ✓ Каждый столик вмещает минимум 2 гостей (фильтр guests=2)
 - ✓ Есть хотя бы один свободный столик
- 09. Создание бронирования
POST .../api/v1/reservations [201 Created, 901B, 5ms]
 - ✓ Статус 201 Created
 - ✓ Бронирование создано корректно
- 10. Просмотр созданного бронирования
GET .../api/v1/reservations/3 [200 OK, 896B, 2ms]
 - ✓ Статус 200 OK
 - ✓ Возвращено именно созданное бронирование
 - ✓ Бронирование связано с выбранным рестораном
- 11. История бронирований пользователя
GET .../api/v1/users/me/reservations [200 OK, 937B, 2ms]
 - ✓ Статус 200 OK
 - ✓ Ответ постраничный
 - ✓ История содержит созданное бронирование
- 12. Создание отзыва о ресторане

```
POST .../api/v1/reviews [201 Created, 657B, 4ms]
✓ Статус 201 Created
✓ Отзыв создан корректно
→ 13. Отзывы выбранного ресторана
GET .../api/v1/restaurants/4/reviews [200 OK, 712B, 2ms]
✓ Статус 200 OK
✓ Ответ содержит список отзывов и средний рейтинг
✓ Отзыв пользователя присутствует в списке
```

	executed	failed
iterations	1	0
requests	13	0
test-scripts	13	0
prerequest-scripts	13	0
assertions	37	0

total run duration: ~0.3s — все проверки пройдены (37/37)

При запуске из Postman GUI результат идентичен: в *Collection Runner* у всех 13 запросов все тесты отмечены зелёным (*PASS*). Повторные прогоны также проходят успешно — за счёт генерации уникального пользователя и случайных даты/времени бронирования конфликтов уникальности и занятости столиков не возникает.

Вывод

Средствами Postman реализован комплексный автотест основного бизнес-процесса приложения бронирования столиков: от регистрации и входа до бронирования столика и публикации отзыва. Коллекция содержит 13 последовательных запросов с 37 проверками (`pm.test` / Chai), охватывающими HTTP-статусы, структуру и типы ответов, бизнес-инварианты и корректную передачу данных между шагами через переменные коллекции; сценарий идемпотентен и запускается как из Postman Collection Runner, так и из CLI (newman). Прогон подтверждает корректную работу API — все проверки пройдены.